

Telegram reconnect storm blocks user logins and code delivery

Incident report · TScopier worker + Telegram auth UI · 2026-09-07

SEVERITY	High	STATUS	Fixed — awaiting deploy
DATES	Reported 2026-09-07 (user Marcin Adamczyk, mrfun1981@gmail.com); second similar report for collinsvaldes63@gmail.com	AFFECTED	4 production users in lockstep (af75b63e , 30c3fa79 , 494bdb70 , dd18ad68)
COMPONENT	worker/src/sessionManager.ts , worker/src/userListener.ts , worker/src/telegramAuthRecovery.ts , src/components/telegram/TelegramConnectFlow.tsx	ROOT CAUSE	Unbounded reconnect + a heal counter wiped every tick → permanent flapping loop; PHONE_CODE_INVALID needlessly fatal

1. Executive summary (plain English)

What this incident is about, in plain English:

- A user told us he could not connect his Telegram account. While investigating, we found the same problem was affecting three other accounts.
- Behind the scenes, each of these accounts' listeners was **flapping** — dropping its connection to Telegram and trying to reconnect every twenty seconds, for hours, without ever settling into a stable "connected" state. A watchdog probe (a lightweight ping to Telegram every 30s) correctly detected that the connections had gone zombie and asked the worker to reconnect — but the reconnect itself had no time limit and got stuck, so every new attempt was handed the same stuck one.
- While a listener was flapping, the app could not even request a fresh login code: the code requests hung and timed out after 90 seconds (this happened 7 times for one user alone).
- Three separate things were wrong: (1) a counting bug meant the safety mechanism that should have restarted a stuck listener never fired; (2) the reconnect step had no time limit, so a network hang could freeze it permanently; (3) a single mistyped login code was treated as a dead end and forced the user to restart the whole login, even though Telegram allows retrying the same code.
- We fixed all three in the worker and improved the on-screen guidance so users know the login code arrives inside the Telegram app, not as a text message.
- Expected result after the fix is deployed: a stuck listener recovers or is cleanly restarted within about a minute instead of flapping for hours, and a mistyped code no longer forces a full restart.

1b. What "flapping" means here

- What TScopier does:** TScopier listens to trading-signal Telegram channels on behalf of each user and copies the signals to their broker. For each connected user, our worker runs a "listener" — a background connection to Telegram that stays open so it can catch new signals the moment they are posted.
- What flapping is:** "Flapping" is when that connection drops and reconnects over and over without ever staying connected. A listener briefly drops for many ordinary reasons (a network blip, Telegram being slow), and it is normal for it to reconnect a second or two later. In this incident the reconnect attempt got *stuck* (it waited forever instead of failing fast), so the listener never actually reconnected, and the twenty-second health-check kept finding it broken and kept kicking it — hours of on/off flapping.
- Why it matters to the user:** While flapping, the user's copier engine looks offline, and — worse — the same stuck reconnect also blocked the login flow, so the user could not even request a new verification code to reconnect their account. Flapping is a *symptom*; the underlying causes are the two bugs in section 4 (a counter that never accumulated, and a reconnect with no time limit).

1c. The story: what happened, in order

- This is the whole incident told as a single, plain-English narrative — from the first report to where things stand now.
1. **The report.** On 2026-09-07 a user, Marcin Adamczyk (mrfun1981@gmail.com), told us he could not connect his Telegram account. A second user reported the same problem. This looked at first like a login problem, so that is where we started looking.
 2. **What we found in the logs.** His login had actually worked earlier that day — at 15:38 he connected successfully using the QR-code method, and our worker began listening to 27 of his channels. But three minutes later the watchdog noticed his connection had gone quiet and asked the worker to reconnect. That reconnect never finished.
 3. **The reconnect got stuck.** A watchdog probe every 30 seconds tests the connection by calling updates.getState — a lightweight ping to Telegram. Two consecutive failures mean the socket is a zombie: it looks open, but Telegram is not responding. The probe failed twice, so the watchdog asked the worker to reconnect. That reconnect tried to open a fresh connection to Telegram, but Telegram did not answer, and the code had no time limit, so it waited forever. From that moment the listener was stuck half-connected: the health-check ran every twenty seconds, found it still broken, and tried again — but every "try again" was handed the same stuck attempt instead of starting a fresh one. The account flapped like this for hours.
 4. **The safety net was also broken.** Normally, after about three failed reconnect attempts the worker gives up on that listener and restarts it cleanly, which would have fixed everything within a minute. But a counting bug meant the failure counter was reset to zero on every check, so the count could never reach three and the restart never happened.
 5. **Login then became impossible.** While the listener was stuck, any attempt to request a new login code hung too — the worker could not reach Telegram at all — and timed out after 90 seconds. This happened seven times. So the user was caught in the worst of both worlds: he was not connected, and he could not reconnect.
 6. **It was never just him.** The same flapping was happening to other accounts in lockstep, which told us the cause was on our worker's side, not something the user had done. As of 2026-09-08, six accounts were flapping and the storm was still live in production.
 7. **Separately, the login flow punished a simple typo.** Even when a code did arrive (inside the Telegram app, not as an SMS), one wrong digit destroyed the whole login and forced a fresh start — even though Telegram allows retrying the same code. This made an already confusing experience much more painful.
 8. **What we fixed.** (a) The failure counter now accumulates while a listener is broken, so a stuck listener is cleanly restarted after about a minute. (b) Every step of reconnecting now has a 45-second time limit, so nothing can hang forever — and stopping a listener can no longer be blocked either. (c) A wrong login code is now retryable instead of fatal. (d) The on-screen guidance now makes it unmissable that the code arrives inside the Telegram app, and always offers a "Send a new code" action.
 9. **Where it stands now.** The fixes are written, tested (48 worker tests, 2 frontend tests, typechecks and lint all green), reviewed, and committed to the staging branch. They are not yet deployed, so the storm is still affecting production. The moment the worker deploys to staging (then production), the flapping should stop and affected users should be able to connect again — using the QR method, or by phone and checking the Telegram app for the code.

2. Issue encountered

- 2026-09-07, production: user mrfun1981@gmail.com (Marcin Adamczyk, Supabase id af75b63e-7202-4b40-a7bf-2115535dcea1) reported he could not connect his Telegram account. A second user(collinsvaldes63@gmail.com) reported the same symptom.
- Investigation of production Railway logs showed the same four users (af75b63e , 30c3fa79 , 494bdb70 , dd18ad68) emitting [sessionManager] listener disconnected but renewing lease anyway every ~20 seconds for hours — a reconnect storm that never healed on its own. The storm is **still ongoing at the time of writing** (2026-09-08), and has since spread to 6 users (995e7936 , 7fd2f7b4 joined) — confirming the fix must be deployed to stop it.
- While flapping, every send_code request from these users hung and timed out after 90 seconds with AUTH_OPERATION_TIMEOUT (observed 7 times for af75b63e between 16:13 and 19:08 UTC).
- The app returned delivery=app on every successful code request: the login code goes to the user's Telegram app, not to SMS, and Telegram offered no fallback (nextDelivery=absent ,

canResend=false).

2b. Evidence from production logs

The following are verbatim excerpts from production Railway logs (environment 25cc1235 , filter af75b63e). Timestamps are UTC.

1. The reconnect storm (sample of the 20-second cadence):

[2026-09-07 19:57:12] [sessionManager] listener disconnected but renewing lease anyway user=af75b63e-... – kicking reconnect in background
[2026-09-07 19:57:30] [sessionManager] listener disconnected but renewing lease anyway user=af75b63e-... – kicking reconnect in background
[2026-09-07 19:57:49] [sessionManager] listener disconnected but renewing lease anyway user=af75b63e-... – kicking reconnect in background
[2026-09-07 19:58:09] [sessionManager] listener disconnected but renewing lease anyway user=af75b63e-... – kicking reconnect in background
[2026-09-07 19:58:32] [sessionManager] listener disconnected but renewing lease anyway user=af75b63e-... – kicking reconnect in background
The same line repeats every ~20s for hours. As of 2026-09-08 06:32Z it is still repeating (51+ occurrences in the sampled window for this user alone).

2. Codes were issued but the user never entered one (delivery = Telegram app, no fallback):

[2026-09-07 14:53:21] [authService] event=send_code_api_ok userId=af75b63e-... responseType=auth.SentCode delivery=app deliveryType=auth.SentCodeTypeApp
nextDelivery= nextDeliveryType=absent timeoutPresent=false canResend=false codeLength=5
[2026-09-07 14:56:10] [authService] event=send_code_api_ok userId=af75b63e-... delivery=app ... nextDeliveryType=absent ... canResend=false
[2026-09-07 15:38:09] [authService] event=send_code_api_ok userId=af75b63e-... delivery=app ... nextDeliveryType=absent ... canResend=false
No verify_code / sign_in event exists for this user — the code was never entered.

3. The watchdog correctly detected a zombie, but the reconnect itself got stuck:

[2026-09-07 15:41:00] [watchdog] probe failed (1/2) for af75b63e-...: 401: AUTH_KEY_UNREGISTERED (caused by updates.getState)
[2026-09-07 15:41:30] [watchdog] probe failed (2/2) for af75b63e-...: 401: AUTH_KEY_UNREGISTERED (caused by updates.getState)
[2026-09-07 15:41:30] [userListener] force reconnect for af75b63e-... reason=watchdog cycle=0286e97f – LAST force reconnect ever logged for this user
[2026-09-07 15:49:59] [watchdog] probe failed (1/2) for af75b63e-...: Cannot send requests while disconnected. Please reconnect.
[2026-09-07 15:50:29] [watchdog] probe failed (2/2) for af75b63e-...: Cannot send requests while disconnected. Please reconnect.
[2026-09-07 15:50:59] [watchdog] probe failed (3/2) for af75b63e-...: Cannot send requests while disconnected. Please reconnect.
[2026-09-07 15:51:29] [watchdog] probe failed (4/2) for af75b63e-...: Cannot send requests while disconnected. Please reconnect.
[2026-09-07 15:51:59] [watchdog] probe failed (5/2) for af75b63e-...: Cannot send requests while disconnected. Please reconnect.
[2026-09-07 15:52:29] [watchdog] probe failed (6/2) for af75b63e-...: Cannot send requests while disconnected. Please reconnect.
[2026-09-07 15:52:59] [watchdog] probe failed (7/2) for af75b63e-...: Cannot send requests while disconnected. Please reconnect.

The auth key was revoked (AUTH_KEY_UNREGISTERED), the watchdog caught it in two probes (30s each), and triggered a force reconnect. But client.connect() had no timeout and Telegram did not answer, so the reconnect hung forever. After 15:41:30 no new force reconnect ever ran — the same stuck promise was returned to every subsequent request. The client was now fully dead (Cannot send requests while disconnected), probes kept failing (counting up to 8/2 and beyond), but nothing could restart it.

4. The user's only successful connection was via QR, then the reconnect wedged:

[2026-09-07 15:38:58] [authService] QR signIn completed user=af75b63e-... – calling finalizeAuth
[2026-09-07 15:39:00] [authService] event=finalize_auth_complete userId=af75b63e-... sessionId=[redacted]
[2026-09-07 15:41:30] [userListener] force reconnect for af75b63e-... reason=watchdog cycle=0286e97f – LAST force reconnect ever logged for this user
After 15:41:30 no force reconnect ever runs again for this user — the wedged reconnectInFlight made every later reconnect request a no-op (it returned the same stuck promise).

5. Login-code requests then hung and timed out at 90s (7 occurrences):

[2026-09-07 16:14:40] [authService] event=send_code_timeout userId=af75b63e-... totalTimeMs=90181
[2026-09-07 16:16:20] [authService] event=send_code_timeout userId=af75b63e-... totalTimeMs=90168
[2026-09-07 18:09:46] [authService] event=send_code_timeout userId=af75b63e-... totalTimeMs=90172
[2026-09-07 18:11:23] [authService] event=send_code_timeout userId=af75b63e-... totalTimeMs=90164
[2026-09-07 19:04:49] [authService] event=send_code_timeout userId=af75b63e-... totalTimeMs=90183
[2026-09-07 19:08:34] [authService] event=send_code_timeout userId=af75b63e-... totalTimeMs=90173
[2026-09-07 19:10:20] [authService] event=send_code_timeout userId=af75b63e-... totalTimeMs=90164
Each attempt never reached Telegram — the worker could not connect at all.

6. Storm breadth (sample counts, 2026-09-08 ~06:30Z):

af75b63e x18 494bdb70 x17 dd18ad68 x17 30c3fa79 x17 995e7936 x17 7fd2f7b4 x17
Six accounts flapping in lockstep, several hours after the report — a worker-wide symptom, not a user-specific mistake.

3. Affected user(s)

USER	SUPABASE ID	WHAT WAS OBSERVED
mrfun1981@gmail.com (Marcin Adamczyk)	af75b63e-7202-4b40-a7bf-2115535dcea1	Connected via QR at 15:38; session later dropped; reconnect wedged; flapped for hours; 3 codes issued (never entered), 7 timeouts
collinsvaldes63@gmail.com	215b1c69-2a93-4f09-b07d-6e04840f2038	4 codes issued with delivery=app , never entered; pending auth expired 4 times; QR login also failed with "Cannot send requests while disconnected"
3 other users	30c3fa79 , 494bdb70 , dd18ad68	Same reconnect storm in lockstep (worker-wide symptom, not user-specific)
2 more users	995e7936 , 7fd2f7b4	Joined the same storm by 2026-09-08 (6 accounts flapping in total)

No trades were lost — trade execution runs on a separate service. The impact was signal-listening downtime and blocked Telegram logins for the affected accounts.

4. Root cause

How the system is meant to work: Every twenty seconds the worker checks each user's listener and renews a "lease" that says "I am the copy handling this Telegram account, nobody else should touch it." If a listener is not connected, the worker tries to reconnect it; and after a few failed tries it is *supposed* to tear the listener down and start it fresh. Two bugs defeated that plan.

Each cause, in plain English and then in technical detail:

1. The "how many times has it failed" counter was being reset every tick, so the restart safety net never fired.

Plain English: The worker was counting how many times in a row the listener had failed to reconnect, planning to restart it after about three failures (~60 seconds). But because of a misplaced line, the counter was cleared back to zero on every single check — even while the listener was still broken. So the count could never reach three, the restart never happened, and the listener flapped forever.

Technical: In `worker/src/sessionManager.ts`, `renewOneListenerLease` increments `disconnectedRenewTicks` when the listener is down and, after `disconnectedRenewHealTicks()` (default $3 \approx 60$ s) consecutive failures, hard-resets the listener (stop + `syncSessions` restart). But line 408 executed `this.disconnectedRenewTicks.delete(userId)` *unconditionally* on every tick — including while disconnected — so the counter never accumulated past 1 and the hard-reset never fired. The lease kept being renewed, so no other worker replica could take the user over either.

2. The reconnect step had no time limit, so a network hang froze it forever — and every later retry was handed the same frozen attempt.

Plain English: When the worker tried to reconnect a listener, it opened a connection to Telegram and waited. If Telegram did not answer, the code just sat there waiting — there was no "give up after 45 seconds" limit. Once that happened, every later reconnect request was given the same stuck attempt instead of trying again, so no new reconnect ever actually ran. This is why the flapping never healed on its own.

Technical: In `worker/src/userListener.ts`, `forceReconnect` called `client.connect()` and the `updates.getState()` probe with no timeout. A hung MTPROTO socket (Telegram DC unreachable) left the `reconnectInFlight` promise pending forever. `requestReconnect` returns the existing in-flight promise when one is set (line ~4508), so every subsequent reconnect request became a no-op and no new `force_reconnect` ever ran — confirmed: the last `force_reconnect` log for `af75b63e` was 15:41:30. `stop()` also awaited `reconnectInFlight` with no bound, so a wedged reconnect blocked `disconnectTelegramSession` and the lease hard-reset too.

3. Two knock-on hazards made things worse: a timed-out connect could double-connect, and a stopped listener could be re-connected.

Plain English: Once we add a timeout (see section 5), two secondary dangers appeared. First, if a connection attempt timed out but was still finishing in the background, the next retry could open a second connection on the same Telegram account at the same time — Telegram treats that as a conflict and can drop the session. Second, if a user disconnected their account while a reconnect was mid-flight, the worker could finish reconnecting a listener that had already been told to stop.

Technical: The retry loop could call `client.connect()` again while the first (timed-out but still running) connect completes in the background, risking a double connection on the same auth key (`AUTH_KEY_DUPLICATED`). The loop also only checked `stopping` at the top of each iteration, so a `stop()` could return while the loop was still mid-connect and re-establish a live socket on a stopped listener.

4. A single mistyped login code destroyed the whole login, even though Telegram allows retrying it.

Plain English: If a user typed the verification code with one wrong digit, our system treated it as a dead end, wiped the login in progress, and forced the user to request a brand-new code and start over. Telegram actually keeps the login valid across wrong attempts, so this made a trivial typo much more painful than it needed to be — and combined with the code arriving in the Telegram app (not SMS), it was a real source of user frustration.

Technical: `worker/src/telegramAuthRecovery.ts` classified `PHONE_CODE_INVALID` as fatal, destroying the pending auth and forcing a full `send_code` restart. Telegram keeps the `phoneCodeHash` valid across wrong-code retries; the code is only invalidated by expiry or a fresh `send_code`.

5. The on-screen guidance was too easy to miss when the code arrives inside the Telegram app.

Plain English: When Telegram delivers the login code inside the app itself (which is what it did for these accounts) rather than by SMS, the screen only showed a small line of text saying so. And if that code expired, the user was left on the code screen with no obvious "send a new code" action, unsure what to do next.

Technical: In `src/components/telegram/TelegramConnectFlow.tsx`, when `codeDelivery === 'app'`, the only guidance was a small hint line; after a code expired there was no resend/new-code affordance on the code screen (resend button only showed when `canResend`).

Why the user-visible symptom has happened before with different root causes: This is the same class of incident (users unable to connect Telegram, `delivery=app` codes never entered) that previously produced write-ups on 2026-08-14, 2026-08-19 and 2026-09-01. Each time the underlying mechanism was different. The 2026-09-01 "Ruth" fix (`2d4c5cb0`) addressed a stuck `authInFlight` in the `auth` path (see section 8). The flapping loop described here lives in the *listener reconnect* path and was not covered by that fix.

5. The fix

Worker + frontend changes; committed to `staging` (not yet deployed at time of writing). Each fix, in plain English and then technical detail:

1. Let the restart safety net actually fire.

Plain English: We moved the "reset the failure counter" step so it only runs when the listener is genuinely back online. Now the count of consecutive failures builds up while the listener is broken, and after about three failures the worker tears the listener down and starts it fresh — exactly as it was always meant to.

Technical: `worker/src/sessionManager.ts` — moved `disconnectedRenewTicks.delete(userId)` into the `connected ( else )` branch so the counter accumulates across disconnected ticks and the hard-reset fires after `disconnectedRenewHealTicks()` failed ticks.

2. Put a time limit on every reconnect step.

Plain English: Every step of reconnecting a listener now has a 45-second cap (configurable). If Telegram does not answer within that time, the attempt is abandoned and treated as failed instead of hanging forever — so a network problem can no longer freeze the listener or block the login flow. Stopping a listener can no longer be blocked by a stuck reconnect either.

Technical: `worker/src/userListener.ts` — added `withTelegramTimeout()` (async race helper) and `telegramConnectTimeoutMs()` (env `TELEGRAM_CONNECT_TIMEOUT_MS`, default 45s, clamped 5–120s). Wrapped `client.connect()`, the `updates.getState()` probe, `disconnect()`, and `stop()`'s await on `reconnectInFlight`.

3. Stop the two knock-on hazards (double-connect and reconnecting a stopped listener).

Plain English: When a connection attempt times out, we now close the socket cleanly before trying again, so a slow background connection cannot sneak in and create a second connection on the same Telegram account. And the reconnect routine now checks "has this listener been told to stop?" after every step, so it can never bring a stopped listener back to life.

Technical: `worker/src/userListener.ts` — added `if (this.stopping) { disconnect; return/break }` after every connect/probe await in both `forceReconnect` and `reconnectAndRetryDialogs`, plus a `disconnect-before-retry` in the transient-error path (prevents `AUTH_KEY_DUPLICATED`). Post-loop warmup also guards on `stopping`.

4. Let a mistyped login code be retried instead of killing the login.

Plain English: A wrong verification code no longer wipes the login in progress. The user can simply re-enter the correct code. Only an expired code (or requesting a fresh code) starts a new login — matching what Telegram actually allows.

Technical: `worker/src/telegramAuthRecovery.ts` — `PHONE_CODE_INVALID` is now recoverable (keeps the pending auth so the user can retry); `PHONE_CODE_EXPIRED` remains fatal.

5. Make it obvious where the login code arrives, and always offer a way forward.

Plain English: When Telegram delivers the code inside the app, the screen now shows a clear, unmissable message: "Open Telegram on your phone ... The code is NOT sent by SMS." And there is now always a "Send a new code" button on the code screen, so if the code expires or is missed, the user has a straightforward action instead of being stuck.

Technical: `src/components/telegram/TelegramConnectFlow.tsx` — prominent callout when `codeDelivery === 'app'` ; always-visible "Send a new code" button (when `canResend` `false`) re-requesting via `send_code` through a new `onRequestNewCode` prop; wired in `CopierEnginePage.tsx` and `TelegramLinkStep.tsx` via a shared `requestCode()` helper.

6. Localized the new text.

Plain English: The new messages are translated for all languages the app supports.

Technical: Added `tgConnectCodeAppHint` + `sendNewCode` keys to `en/es/fr/ar` and `pl/ru/sv/nl/ja` (copierEngine bundle) + `types.ts` .

6. Files changed

FILE	CHANGE
<code>worker/src/sessionManager.ts</code>	Heal counter delete moved into connected branch (hard-reset can now fire)
<code>worker/src/userListener.ts</code>	<code>withTelegramTimeout</code> + <code>telegramConnectTimeoutMs</code> ; connect/probe/disconnect/stop bounded; stopping checks after every await; disconnect-before-retry
<code>worker/src/telegramAuthRecovery.ts</code>	<code>PHONE_CODE_INVALID</code> recoverable; removed from fatal list
<code>worker/src/telegramAuthRecovery.test.ts</code>	Tests updated to assert the new classification
<code>worker/src/withTelegramTimeout.test.ts</code> (new)	3 tests: resolve / timeout-reject / error-reject for the timeout helper
<code>src/components/telegram/TelegramConnectFlow.tsx</code>	Prominent app-delivery callout; "Send a new code" button; <code>onRequestNewCode</code> prop
<code>src/pages/dashboard/CopierEnginePage.tsx</code> , <code>src/pages/onboarding/steps/TelegramLinkStep.tsx</code>	Extracted <code>requestCode()</code> helper, wired to <code>onRequestNewCode</code>
<code>src/i18n/locales/{en,es,fr,ar}.ts</code> , <code>src/i18n/locales/copierEngine/{pl,ru,sv,nl,ja}.ts</code> , <code>types.ts</code>	New <code>tgConnectCodeAppHint</code> + <code>sendNewCode</code> keys

7. Verification

- Worker typecheck (`npm --prefix worker run build / tsc`): PASS.
- Worker tests (`telegramAuthRecovery` , `authService.resend` , `sessionManager.shutdown` , `sessionManager.realtime` , `withTelegramTimeout`): 48 pass / 0 fail.
- Frontend typecheck (`tsc -b`): PASS.
- Frontend component test (`TelegramConnectFlow.test.ts` , vitest): 2 pass / 0 fail.
- ESLint on all changed files: clean. (Pre-existing React-Compiler debt in the two page files is unrelated to this change.)
- Post-implementation review: an automated code review caught a critical defect in the first version of `withTelegramTimeout` (it was not `async` , so the timer was cleared synchronously and the timeout never fired) plus two high-severity hazards (orphaned connect, post-stop reconnect). All were fixed, regression-tested, and re-reviewed → APPROVE_WITH_NOTES, with both optional notes applied.

8. How this ties to the 2026-09-01 fix

In plain English: This is not the first time users could not get a Telegram login code, and it is worth explaining how this incident relates to the fix we made on September 1 ("the Ruth fix"), because the two incidents share a surface symptom but had different underlying causes — and both fixes are needed.

- The September 1 problem:** a login-code request could get stuck waiting to take over the account, and while it was stuck, every new request was refused with "a login is already in progress." That fix added a time limit (90 seconds) and an auto-expiry (2 minutes) to the *login request* part of the flow, so a hung login could never block future logins forever. Those settings are unchanged and still in place today.
- The September 7 problem (this report):** the stuck part was not the login request — it was the *listener reconnect*. The September 1 timeout protected the login path (which is exactly why these users' code requests failed cleanly after 90 seconds instead of hanging), but it did *not* put any time limit on reconnecting the listener itself. That unlimited wait is what wedged the listener and produced the flapping loop.
- Why both fixes are needed:** the September 1 fix keeps login requests from hanging; this fix keeps a broken listener from flapping forever and from blocking new login attempts. Together they cover both halves of the flow.

9. What to expect after deploy

- A disconnected listener now recovers or is cleanly restarted within ~60 seconds (default `LISTENER_DISCONNECT_HEAL_TICKS` = 3 × 20s renew interval), instead of flapping for hours.
- A network hang can no longer freeze reconnect indefinitely — each connect/probe/disconnect step is bounded (45s default), and `stop()` cannot be blocked by a wedged reconnect.
- A mistyped login code no longer destroys the pending login — the user can simply re-enter the code. Only an expired code (or a fresh `send_code`) starts a new login.
- Users connecting by phone see a clear message that the code arrives inside the Telegram app (not SMS) and always have a "Send a new code" action.
- Important: Telegram controls delivery. The login code will still be delivered inside the Telegram app for these accounts (`delivery=app`); we cannot force an SMS. Users must open Telegram on their phone and look for the message from Telegram.

10. How we will test it ourselves

- Unit / type level (already green):** worker `tsc` , the 5 targeted worker test files (48 tests), frontend `tsc -b` , the 2 component tests, and ESLint on changed files.
- Staging deploy:** push `staging` to the upstream Railway branch; confirm Listener/Trade/Backtest deployments build (Docker `npm run build`) and reach `SUCCESS` via the Railway GraphQL API.
- Verify the four flapping users stop flapping:** after deploy, watch production Railway logs for `af75b63e` , `30c3fa79` , `494bdb70` , `dd18ad68` . The `listener disconnected` but `renewing lease` anyway cadence should stop; any disconnected listener should be hard-reset once and then stay connected (or be restarted cleanly by `syncSessions`).
- Timeout behavior:** with `TELEGRAM_CONNECT_TIMEOUT_MS` set low in a staging env, force a hung connect (e.g. block outbound MTPROTO) and confirm the reconnect cycle logs a timeout and retries/exhausts instead of wedging `reconnectInFlight` , and that `stop()` / `disconnectTelegramSession` still completes.
- Auth retry behavior:** in a staging session, call `verify_code` with a wrong code and confirm the error surfaces to the UI and the pending auth survives for a retry; confirm `PHONE_CODE_EXPIRED` still starts a fresh login.
- UI:** open the connect flow in a browser with a phone login, confirm the "code arrives in the Telegram app" callout and the "Send a new code" button appear, and that QR fallback still shows.

11. Deployment status

Not yet deployed — the storm is live in production right now. The fix is committed to the `staging` branch (frontend + worker) but has not shipped. Until it deploys, the affected users

continue to flap. Deployment requires: (1) push `staging` to the upstream Railway branch, (2) confirm the listener/trade worker deployments succeed, (3) after validation on staging, propagate to `main` and production. The frontend changes additionally require a Netlify redeploy (env vars are baked at build time).

12. Follow-ups

- Deploy worker to Railway staging, then prod; deploy frontend to Netlify.
- Re-check `mrfun1981@gmail.com` and `collinsvaldes63@gmail.com`: after deploy they should be able to reconnect via QR (or phone, checking the Telegram app for the code). Their `telegram_account_claims` rows are intact; a new session is created on successful connect.
- Consider raising `LISTENER_DISCONNECT_HEAL_TICKS` if a slow-but-legitimate reconnect (>60s) is observed being hard-reset too eagerly.
- Consider alerting on repeated `listener disconnected but renewing lease anyway` so a future storm is caught before users report it (same lesson as the 2026-08-24 realtime incident).
- Investigate the worker-instance/DC connectivity that exposed the bug (4 users affected in lockstep) — the reconnect-timeout fix bounds the damage but the underlying network cause should be understood.